

A&D; Blood Pressure Monitor

Integration Status Report

Prepared by: VAYUZ Technologies, Noida | Date: 28 September 2026 | Project: Awesom Living

1. Executive Summary

This report covers the integration status of the **A&D UA-656BLE** Blood Pressure Monitor into the Awesom Living elderly care platform. The device communicates over Bluetooth Low Energy (BLE) and is designed to automatically transmit readings to the Raspberry Pi hub after each measurement.

Component	Status	Notes
Backend API (Cloud)	DONE	Device model, registration, data routing — fully built
Pi Bridge Daemon (bp_bridge.py)	DONE	v15 — BLE scanner, data parser, offline queue
Pi Provisioning (bp_provision.py)	DONE	Scan, pair, bond, buffer config — fully built
Pi Dashboard UI (Scan & Pair)	DONE	Scan / Pair & Map buttons wired to backend
BLE Pairing (Cuff ↔ Pi)	BLOCKED	Cuff not pairing reliably — see Section 3
End-to-End Data Flow	NOT TESTED	Blocked by pairing — no live readings received yet
Mobile App (BP Card)	NOT STARTED	Waiting for successful end-to-end data

2. What Has Been Built

2.1 Cloud Backend (Fully Complete)

- Device Model:** BpMonitor device type added to the platform with MAC address and serial number fields.
- Device Registration:** API endpoint registers the cuff by BLE MAC address, with automatic deduplication (same MAC + same resident = reactivate existing record).
- Data Routing:** BP readings follow the resident-mapped path (same as the GLK sleep sensor) — the cuff is linked directly to the resident, not to the home.
- Online Status:** The system checks the last received log to determine if the cuff is online (same pattern as GLK).

2.2 Raspberry Pi — Bridge Daemon (bp_bridge.py v15)

A production-ready BLE bridge daemon that runs continuously on the Pi via pm2. Based directly on A&D's official reference implementation (ad_bridge.py).

- **Continuous BLE Scanner:** Uses Bleak library with detection callbacks — reacts instantly when the cuff starts advertising after a measurement.
- **A&D Protocol Compliance:** Writes DateTime (0x2A08) and enables indications on Blood Pressure Measurement (0x2A35) within the required 5-second window after encryption.
- **IEEE 11073 SFLOAT Decoder:** Correctly decodes systolic, diastolic, MAP, pulse rate, and irregular heartbeat flag from the BLE indication payload.
- **NaN Marker Filtering:** Filters out A&D's special NaN marker frames (SFLOAT 2047) and measurement error frames.
- **Offline Queue:** If the backend is unreachable, readings are persisted to disk and flushed oldest-first when connectivity returns.
- **Cooldown Logic:** 60-second cooldown per device to prevent duplicate reads.
- **Data Pipeline:** Cuff → BLE → bp_bridge.py → HTTP POST to Pi server (/api/bp/reading) → Cloud backend (/api/bp/log).

2.3 Raspberry Pi — Provisioning (bp_provision.py)

A CLI tool called by the Pi dashboard's server.js to handle BLE scanning and pairing. Exposed via two API endpoints on the Pi:

- **POST /api/bp/scan:** Scans for nearby BLE devices advertising the Blood Pressure Service (UUID 0x1810). Returns device name, MAC address, and signal strength (RSSI).
- **POST /api/bp/pair:** Connects to the selected cuff, performs BLE bonding, writes DateTime, configures the memory buffer to store up to 200 readings, and trusts the device in BlueZ.
- **Combined scan_pair mode:** A single-operation mode that scans and pairs immediately on detection — eliminates the timing gap where the cuff stops advertising.
- **Stale Bond Cleanup:** Automatically clears old BlueZ bonds before re-pairing (bluetoothctl remove/untrust).
- **D-Bus Agent:** Registers a NoInputNoOutput BlueZ agent to handle the cuff's BLE Security Request during pairing.
- **Error Handling:** Detects ERR 10 (bond key mismatch), timeout, and advertising-window-expired scenarios with user-facing guidance.

2.4 Pi Dashboard UI

The Pi's local web dashboard has a BP monitor section with Scan and Pair & Map buttons that call the provisioning endpoints. The scan results show the cuff's name and MAC address. The Pair & Map button triggers the pairing flow and registers the device on the cloud backend.

3. Current Blocker: BLE Pairing Reliability

The core issue is that the A&D UA-656BLE cuff does not pair reliably with the Raspberry Pi's BLE adapter. The software is fully built and functionally correct, but the BLE pairing step between the physical cuff and the Pi fails intermittently.

3.1 How the Cuff Works (A&D Protocol)

The A&D UA-656BLE is a **push-mode** device. After the user takes a blood pressure measurement, the cuff automatically advertises over BLE for approximately 60 seconds. During this window, the Pi's bridge daemon must:

- **Step 1:** Detect the BLE advertisement
- **Step 2:** Connect and complete BLE encryption
- **Step 3:** Write the current DateTime to characteristic 0x2A08 (within 5 seconds of encryption)
- **Step 4:** Enable indications on Blood Pressure Measurement characteristic 0x2A35
- **Step 5:** Receive the buffered readings as BLE indications

The cuff supports **only one bonded master** at a time. If it was previously paired to a phone or another device, it will reject the Pi's connection (ERR 10 on the cuff display). This is a hardware-level limitation of the A&D BLE protocol.

3.2 Issues Encountered During Pairing

Issue	Description	What We Tried
ERR 10 (Bond Key Mismatch)	Cuff rejects Pi's connection because it has stale bond keys from a previous pairing (phone or earlier Pi attempt).	Remove cuff batteries for 30s to clear bond memory. Hold START until 'Pr' blinks. Clear Pi-side bond via bluetoothctl remove.
Pr Mode Timing Window	Cuff advertises for only ~30–60s after entering Pr mode. If the scan and pair are separate API calls, the window expires before pairing starts.	Built combined scan_pair mode that scans and connects in one operation. Detection callback fires instantly on advertisement.
BlueZ Stale Cache	Linux BlueZ daemon caches device state from previous failed attempts. Stale cache causes connection failures on retry.	Added bluetoothctl remove + untrust before every pair attempt. Adapter power-cycle between retries as last resort.
BLE Connection Timeout	connect() call times out even when the cuff is visible in scan results. Occurs more frequently after failed attempts.	Increased timeout to 30s. Added retry logic (2 attempts with adapter reset between). Fresh BLEDevice object from targeted re-scan.
D-Bus Security Request Race	Cuff sends BLE Security Request before Bleak's pair() registers its agent, causing authentication to fail.	Pre-register a NoInputNoOutput D-Bus agent via background bluetoothctl process before connection attempt.
Adapter Instability After Failures	Repeated failed pairing attempts leave the Pi's Bluetooth adapter in an unstable state.	hciconfig hci0 reset as recovery step. Adapter health check (bluetoothctl show) before each attempt.

3.3 Iterations of the Provisioning Code

The provisioning script has gone through multiple iterations to address each failure mode. Key changes across versions:

- **Early versions:** Used subprocess calls to bluetoothctl for scanning and pairing. Failed due to timing issues and unreliable output parsing.

- **Mid versions:** Switched to Bleak (Python BLE library) for direct BLE control. Added retry logic and adapter reset. Improved but still intermittent failures.
- **Current version:** Combined scan+pair in a single operation with detection callbacks, D-Bus agent pre-registration, stale bond cleanup, and structured error messages. Based directly on A&D's reference implementation (ad_pair.py).

The bridge daemon (bp_bridge.py) has similarly been rewritten to v15, removing complex subprocess architecture in favour of A&D's proven single-process scanner-connect-read loop.

4. Root Cause Analysis

The BLE pairing issues are not caused by a software bug. The underlying challenges are:

- **Single-Master Limitation:** The A&D UA-656BLE supports only one bonded master. Any previous pairing (even a brief test from a phone) leaves bond keys that block the Pi. The cuff must never be paired to any phone — the Pi must be the exclusive master.
- **Short Advertising Window:** After entering Pr mode, the cuff advertises for a limited time (~30–60s). In a field environment, the time between the user pressing 'Scan' on the dashboard and clicking 'Pair' can exceed this window.
- **BlueZ + Bleak Stack Complexity:** The Linux Bluetooth stack (BlueZ) combined with the Python Bleak library introduces edge cases around bond key management, D-Bus agent registration, and adapter state that do not occur on mobile platforms.
- **No Vendor Debug Support:** A&D provides a reference Python script (ad_pair.py / ad_bridge.py) but limited debugging support for Linux-specific BLE issues. The reference code assumes a clean environment with no prior bond history.

5. What Is Needed to Unblock

5.1 Immediate Requirements

- **Clean Cuff State:** The cuff must be factory-reset (batteries removed for 30 seconds, reinserted, START held until 'Pr' blinks). It must have NO prior bond with any phone.
- **Dedicated Test Environment:** A controlled setup with the Pi, the cuff, and no other BLE devices interfering. The Pi's Bluetooth adapter should be in a fresh state.
- **Field Visit / On-Site Testing:** BLE pairing issues are highly environment-sensitive (distance, interference, adapter state). A hands-on session where the engineer can see the cuff display (Pr, ERR codes) while monitoring the Pi's BLE logs in real-time is essential.

5.2 Recommended Next Steps

#	Action	Owner	Timeline
1	Factory-reset the cuff (clear all bonds). Ensure it has NOT been paired to any phone.	Field Team	Before test
2	On-site test session: scan, pair, take a reading, verify data reaches backend.	Backend + Field	1 day
3	If pairing succeeds: run bp_bridge.py via pm2 and verify automated reading capture over 3 days.	Backend	3 days

#	Action	Owner	Timeline
4	If pairing fails: capture full BLE debug logs (BP_LOG_LEVEL=DEBUG) and share with VAYUZ for analysis.	Field Team	Same day
5	Once end-to-end data flow is confirmed: build the mobile app BP card and notifications.	Frontend	Sprint 3

6. Technical Architecture (Data Flow)

The complete data pipeline from the BP cuff to the mobile app:

Stage	Component	Detail
1. Measurement	A&D UA-656BLE Cuff	User takes BP reading. Cuff stores result and starts BLE advertising.
2. BLE Transfer	bp_bridge.py (Pi)	Detects advertisement, connects, syncs DateTime, receives readings via BLE indications.
3. Local Relay	Pi server.js	Receives reading via HTTP POST at /api/bp/reading. Forwards to cloud backend.
4. Cloud Storage	Cloud Backend	Receives reading at /api/bp/log. Stores in database. Links to resident via device MAC.
5. Mobile App	React Native App	Displays BP history, trends, and alerts on the family member's phone. (Not yet built)

7. Key Constraints

- The cuff must **never** be paired to any phone (including the A&D Connect app). The Raspberry Pi is the only permitted master.
- The cuff supports only Bluetooth Low Energy (BLE) — it cannot transmit data over WiFi or any other protocol.
- Readings taken while the Pi bridge is offline are stored in the cuff's internal memory (up to 200 readings) and will be transmitted on the next successful BLE connection.
- The Raspberry Pi must be within BLE range (~10m line of sight) of the cuff for data transfer.